Corso di Fondamenti di Informatica 2
CdL Ingegneria Informatica
Ing. Franco Zambonelli

ARCHITETTURA DEGLI ELABORATORI: CONCETTI GENERALI

Lucidi Realizzati in Collaborazione con:

Prof. Letizia Leonardi
Università di Modena

Prof. Antonio Corradi
Università di Bologna

ARCHITETTURA DEGLI ELABORATORI ELETTRONICI DIGITALI

Elaborazione di dati memorizzati in forma digitale:
- caratteri alfabetici,
- valori numerici interi o reali, ecc..

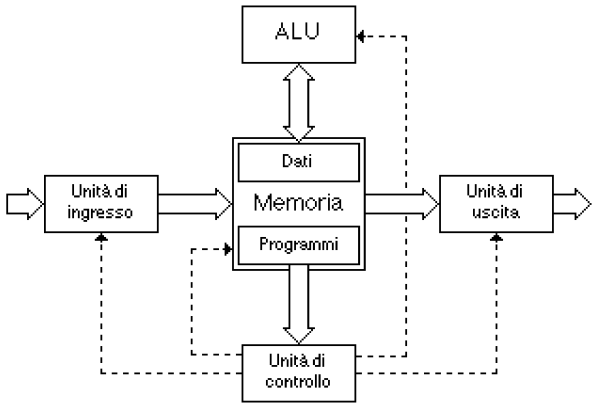
Il sistema **digitale** universalmente adottato è
il **sistema binario** $\Rightarrow$ 0 e 1 - **bit** (da **BI**nary digi**T**)

Operazioni elementari:
- operazioni aritmetiche
- operazioni logiche

Circuiti **elettronici** $\Rightarrow$ semplici elementi circuitali
- funzioni **and**, **or** e **not** dell'algebra booleana

Un elaboratore è in grado di eseguire
una qualsiasi attività (purchè computabile)
in base a **programmi memorizzati**

Dati e **istruzioni**
- risiedono **in memoria**
- possono essere **modificati**



- **unità di ingresso/uscita**
- **memoria** $\Rightarrow$ gerarchia di memorie
- **ALU** (**A**rithmetic-**L**ogical **U**nit)
  - operazioni aritmetiche (somme, sottrazioni, ...)
  - operazioni logiche (and, or, not, ...)
  - operazioni di spostamento di bit (shift, rotate, ...)
- **unità di controllo**
  - interpreta le istruzioni del programma
  - controlla l'esecuzione delle istruzioni

Central **P**rocessing **U**nit (o **processore**)
CPU = ALU + Unità di controllo + Registri + ...

Bus di comunicazione
- informazioni (dati e/o istruzioni)
- segnali di controllo

linee elettriche $\equiv$ bit da trasferire

MEMORIA

Più memorie realizzate con dispositivi molto diversi
- sia come struttura fisica
- sia come tecnica di memorizzazione impiegata

Gerarchia di memorie:
- dalle memorie più veloci, ma molto costose e con bassa capacità di memorizzazione
- alle memorie più lente, ma meno costose e con alta capacità di memorizzazione.

Velocità = tempo necessario per leggere o scrivere un'informazione nella memoria stessa (*tempo di accesso*)

Capacità di memorizzazione = numero di bit (o byte) memorizzabili

Costo = costo globale della memoria / capacità

Tipo di accesso ai dati
- sequenziale
- diretto o casuale (random)
- semi-diretto o semi-casuale (semi-random)

❶ REGISTRI MACCHINA

- contenuti nella CPU
- estremamente veloci
- accesso immediato: direttamente utilizzabili in lettura e scrittura dall'ALU
- capacità di pochi bit (di norma da 8 a 64)
- memorizzano temporaneamente dati e/o istruzioni

- **registri generali** (utilizzabili in ogni istruzione)
- **registri dedicati** (con compiti specifici)

❷ MEMORIA CACHE

- contenuta nella CPU, oppure esterna (II livello)
- memoria di transito tra registri e memoria principale
- rende più veloce l'accesso ai dati e/o alle istruzioni

❸ MEMORIA PRINCIPALE (o **centrale**, o di **lavoro**)

- deve contenere dati e programmi da eseguire
- sufficientemente veloce
- buona capacità di memorizzazione - sino a centinaia di Mb
- accesso diretto ai suoi elementi

- costituita da N elementi, detti parole (o locazioni, o celle), con N in genere potenza di 2

- ogni parola:
  - contiene L bit, con L in genere multiplo di 8
  - è direttamente accessibile mediante un indirizzo univoco compreso tra 0 e N-1
  - può contenere indifferentemente un dato o un'istruzione (o parte di essi)

❹ MEMORIE SECONDARIE

- memorie permanenti
- più lente della memoria principale
- alta (o altissima) capacità di memorizzazione - sino a decine di Gb

- dischi magnetici - accesso semi-random
- nastri magnetici - accesso sequenziale

BUS ESTERNI

Collegano
CPU - Memoria principale - Dispositivi di I/O

Scambio di dati tra CPU e periferia mediante **tre bus**:
- **bus degli indirizzi** (*address bus*) - CPU → periferia
- **bus dei dati** (*data bus*) - CPU ↔ periferia
- **bus di controllo** (*control bus*) - linee di controllo

Durante il **trasferimento di un dato**:
- **bus degli indirizzi** - indirizzo parola di memoria o porta di I/O interessata al trasferimento
- **bus dei dati** - dato da trasferire
- **bus di controllo** - segnali di controllo necessari a garantire la corretta esecuzione del trasferimento

DIMENSIONAMENTO DEL BUS DEGLI INDIRIZZI

Indirizzamento di
- tutte le parole di memoria principale
- tutte le porte di I/O

Per indirizzare 2^{20} = 1024K parole di memoria è necessario un bus degli indirizzi di **20** bit

DIMENSIONAMENTO DEL BUS DEI DATI

Dipende
- dalla dimensione della parola di memoria
- dal tipo di istruzioni che la CPU può eseguire

Per accedere ad una memoria con parole di 8 bit:
- **bus a 8 linee**
- anche **bus a 16, 32 o più linee**

DIMENSIONAMENTO DEL BUS DI CONTROLLO

Dipende dal numero di segnali di controllo che CPU e dispositivi periferici devono scambiarsi

I bus sono **risorse hardware** che
- permettono la comunicazione tra unità diverse
- devono essere gestiti in modo appropriato

- trasferimenti dati tra memoria principale e dispositivi di I/O senza controllo diretto della CPU
- esistenza di più processori con bus in comune: microprocessore + coprocessore matematico

Necessità di un **controllore dei bus** (o *bus arbiter*), in grado di coordinare l'utilizzo dei bus
Sullo stesso chip del µP, oppure su chip separato

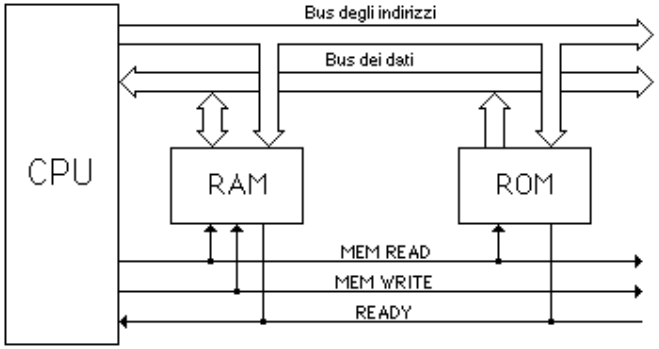
MEMORIA PRINCIPALE

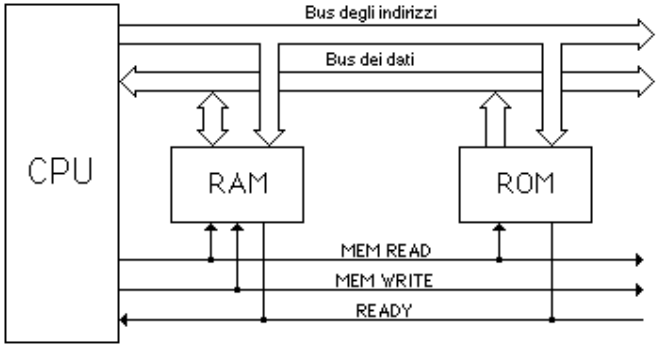
Suddivisa in più ***banchi di memoria***
Ogni singolo banco può essere una RAM, o una ROM
Nelle ROM programmi e/o dati che non devono essere più modificati

OPERAZIONI SULLA MEMORIA

- **lettura** del contenuto di una locazione di memoria
- **scrittura** in una locazione di memoria

Bus degli indirizzi + bus dati +
linee di controllo MEM READ, MEM WRITE e READY





Lettura:

❶ **CPU** - bus degli indirizzi = indirizzo locazione da leggere
❷ **CPU** - richiede una lettura in memoria ⇒ segnale sulla linea MEM READ
❸ **banco di memoria** - legge il dato contenuto nella locazione selezionata e lo carica sul bus dati
❹ **banco di memoria** - indica la presenza del dato sul bus dei dati ⇒ segnale sulla linea READY

Scrittura:

❶ **CPU** - bus degli indirizzi = indirizzo locazione in cui scrivere; bus dei dati = dato da scrivere
❷ **CPU** - richiede una scrittura in memoria ⇒ segnale sulla linea MEM WRITE
❸ **banco di memoria** - memorizza il dato contenuto nel bus dei dati nella locazione selezionata
❹ **banco di memoria** - indica la fine dell'operazione (disponibilità dei bus) ⇒ segnale sulla linea READY

UNITÀ LOGICO-ARITMETICA

Operazioni di manipolazione dei dati:
- operazioni aritmetiche
- operazioni logiche (and, or, not, ...)
- operazioni di spostamento di bit (shift, rotate, ...)

Opportuni comandi, inviati dall'unità di controllo in base all'istruzione corrente, selezionano:
- il tipo di operazione da eseguire
- gli operandi (registro e/o memoria)
- la destinazione del risultato (registro e/o memoria)

☞ La **somma di due interi** di **n** bit produce un risultato di **n+1** bit
Il bit più significativo del risultato, **bit di riporto** (**carry**), deve essere memorizzato in un opportuno registro a 1 bit

☞ La **moltiplicazione di due interi** di **n** bit produce un risultato di **2n** bit
Il risultato deve essere memorizzato in una coppia di registri

UNITÀ DI CONTROLLO

- **interpreta** un'istruzione del programma per volta
- **comanda l'esecuzione** dell'istruzione inviando alle altre unità appropriati segnali di controllo

Accensione della macchina - partenza a freddo (*bootstrap*)
- lettura da una locazione fissa di memoria dell'indirizzo della prima istruzione del programma di bootstrap
- caricamento dell'indirizzo letto nel *program counter* (PC)

Il *program counter* contiene sempre l'**indirizzo della successiva istruzione da eseguire**

Ogni istruzione deve essere:
- recuperata dalla memoria (**fase di fetch**) e quindi
- eseguita (**fase di execute**)

Nella **fase di *fetch***, l'unità di controllo:
- carica nell'*instruction register* (IR) l'istruzione letta in memoria all'indirizzo dato da PC
- incrementa PC, affinchè punti all'istruzione successiva

Nella **fase di *execute***, l'unità di controllo:
- decodifica l'istruzione contenuta in IR
- genera i segnali di controllo necessari per l'esecuzione dell'istruzione:
  - trasferimenti tra registri e/o memoria
  - operazioni che coinvolgono l'ALU
  - operazioni di ingresso/uscita
  - modifica del contenuto di PC (istruzioni di salto)

Ripetizione del ciclo di *fetch* ed *execute*
sino all'esecuzione di una particolare istruzione (**HALT**) che pone l'unità di controllo in uno stato di attesa di un evento esterno

☞ Il contenuto di una parola di memoria può essere considerato:
- un'istruzione
- un dato

La (corretta) interpretazione dipende dal contesto:
- accesso nella fase di fetch ⇒ istruzione
- accesso nella fase di execute ⇒ dato

SINCRONIZZAZIONE

Le attività della CPU devono essere **sincronizzate**

Segnale di temporizzazione (**clock**)
- generato da un apposito generatore di *clock*
- inviato alle varie unità che compongono la CPU

Il *clock*
- è un segnale ciclico, simile a un'onda quadra
- ha una frequenza prefissata e costante nel tempo

- I segnali di controllo (generati dall'unità di controllo) indicano quale azione occorre fare
- Il segnale di *clock* indica il momento esatto in cui eseguire l'azione

Ogni azione è controllata dal *clock*
La **durata di un'azione** si può esprimere in
- **numero di impulsi di *clock*** necessari per il completamento dell'azione

Per **valutare il tempo reale** occorrente per completare un'azione è necessario conoscere la **frequenza del segnale di *clock*** dell'elaboratore

Frequenza f del *clock* in Hz (Herz o cicli al secondo)
Periodo T del *clock* in secondi (intervallo di tempo tra due impulsi successivi di *clock*)

$$f = 1 / T$$

Esempio

Frequenza di 80MHz ⇒ periodo di 12.5ns
Frequenza di 200MHz ⇒ periodo di 5ns
Frequenza di 333MHz ⇒ periodo di 3ns

Per eseguire un'istruzione sono necessari 10 impulsi di *clock*

Tempi di esecuzione reali:
- 125ns su un sistema a 80MHz
- 30ns su un sistema a 333MHz

ISTRUZIONI MACCHINA

- **stringa di 0 e di 1**
- **significato particolare per l'unità di controllo** dell'elaboratore su cui viene eseguita

Ogni CPU:
- ha il proprio insieme di istruzioni macchina (***instruction set***)
- è in grado di eseguire solo le istruzioni appartenenti a tale insieme

Le istruzioni macchina dipendono dalla realizzazione fisica della CPU ⇒
il loro numero e il loro formato variano al variare dell'elaboratore (o della *famiglia* di elaboratori)

Per scrivere programmi in **linguaggio macchina** è necessario:

- conoscere l'architettura interna del processore che si utilizza
- conoscere i dettagli relativi alla scrittura delle singole istruzioni:
  - codici numerici delle istruzioni macchina
  - formato interno delle istruzioni macchina
  - rappresentazione degli operandi, ...
- gestire direttamente gli indirizzi in memoria per
  - riferimenti ai dati
  - salti al codice

LINGUAGGIO ASSEMBLER

Programma scritto in linguaggio assembler
⇒ sequenza di istruzioni elementari
corrispondenza 1:1 con istruzioni macchina

Vantaggio:
utilizzo di **nomi simbolici** al posto di

- codici numerici (*mnemonici*)
- indirizzi di memoria (nomi di variabili e label)

CLASSIFICAZIONE DELLE ISTRUZIONI

- **istruzioni per il trasferimento dati**
  registro/memoria ⇒ registro/memoria:
  - LOAD (da memoria principale a registro)
  - STORE (da registro a memoria principale)
  - MOVE (trasferimento generico)
- **istruzioni aritmetiche**
  operazioni aritmetiche su interi (e reali):
  - somma (ADD) e sottrazione (SUB)
  - incremento (INC) e decremento (DEC) di 1
  - moltiplicazione (MUL) e divisione (DIV)
- **istruzioni per la manipolazione di bit**:
  - operazioni logiche (AND, OR, XOR e NOT)
  - operazioni di scorrimento (SHIFT) e rotazione (ROTATE) di bit
- **istruzioni per il trasferimento del controllo**:
  - salti incondizionati e condizionati (JUMP, BRANCH)
  - chiamate a sottoprogramma (CALL)
  - ritorno da sottoprogramma (RET)

  ☞ uniche istruzioni che modificano PC

  ☞ strutture di controllo dei linguaggi ad alto livello
  ⇒ salti incondizionati e condizionati
- **istruzioni per il controllo del processore**:
  - HALT
- **istruzioni per l'input/output**:
  - IN e OUT

FORMATO DELLE ISTRUZIONI

- istruzione divisa in varie sezioni
- ogni sezione ha un particolare significato per l'unità di controllo

Prima sezione: **codice operativo dell'istruzione**
codice numerico che indica all'unità di controllo quale particolare istruzione deve essere eseguita

Resto dell'istruzione: informazioni necessarie per determinare la collocazione degli **operandi dell'istruzione**

Il termine ***operando*** indica indistintamente:

- un **operando sorgente** da leggere da un registro o dalla memoria
- un **operando destinazione** da memorizzare in un registro o in memoria
- l'**indirizzo dell'istruzione** a cui trasferire il controllo, nel caso di istruzioni di salto

Classificazione in base al numero di **operandi espliciti**:

- **istruzioni a tre operandi**: ad esempio, un'istruzione di somma
- **istruzioni a due operandi**: ad esempio, un'istruzione di trasferimento
- **istruzioni a un operando**: ad esempio, un'istruzione di salto
- **istruzioni senza operandi**: ad esempio, un'istruzione HALT

Utilizzo di **operandi impliciti**

Un'istruzione di somma può utilizzare il registro accumulatore come **sorgente implicita** di uno degli addendi e **destinazione implicita** del risultato

Un'istruzione di trasferimento dalla memoria a un registro macchina può utilizzare il registro accumulatore come **destinazione implicita**

MODALITÀ DI INDIRIZZAMENTO

Accesso alla memoria principale ⇒

indirizzo effettivo (I_{eff})

della locazione di memoria principale coinvolta

Il calcolo di I_{eff} dipende dalla
modalità di indirizzamento utilizzata nell'istruzione

Caso più semplice - **indirizzamento diretto**:
l'indirizzo dell'operando è contenuto nell'istruzione

Tecniche diverse per ritrovare gli operandi
ossia per calcolare I_{eff}

I_{eff} calcolato in funzione di:
- campo operando dell'istruzione
- eventuali registri macchina
- eventuali locazioni di memoria principale

SPAZIO DI INDIRIZZAMENTO

dimensione, in numero di elementi indirizzabili, di
- una **entità *concreta***:
  - memoria principale
  - sistema di I/O, ...
- una **entità *astratta***:
  - memoria utilizzabile da un programma, ...

Entità concreta ⇒ spazio di indirizzamento **fisico**
Entità astratta ⇒ spazio di indirizzamento **logico**

Lo spazio di indirizzamento (fisico) di
una memoria principale è di N parole ⇒
la memoria **può** contenere sino a N parole,
con indirizzi che vanno da 0 a N-1

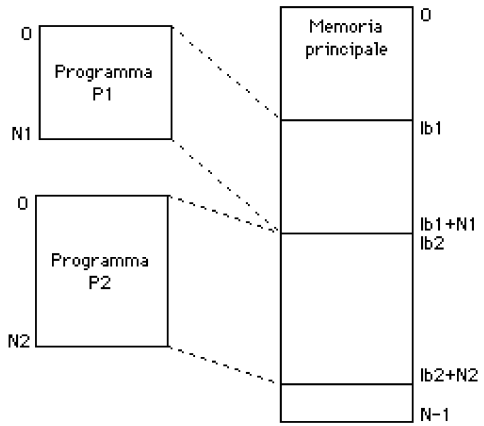
Lo spazio di indirizzamento dell'I/O è di K porte ⇒
possono esistere sino a K dispositivi di I/O,
con indirizzi che vanno da 0 a K-1

Spazio di indirizzamento logico di un programma
- quantità di *memoria astratta* (o *virtuale*) che il programma può utilizzare
- **può essere diverso** dallo spazio di indirizzamento fisico della memoria principale su cui il programma dovrà essere eseguito

L'I_{eff} dell'operando di un'istruzione può essere
- un **indirizzo fisico** (o **assoluto**)
- un **indirizzo logico** (o **relativo**)

Se le istruzioni di un programma contengono *indirizzi fisici*:
- lo spazio di indirizzamento logico del programma **coincide** con lo spazio di indirizzamento fisico della memoria principale
- il programma deve essere **caricato in una ben precisa zona di memoria** principale
- il programma **non può essere spostato**

Se le istruzioni di un programma contengono solo *indirizzi logici*:
- lo spazio di indirizzamento del programma può essere **inferiore, uguale o maggiore** dello spazio di indirizzamento fisico della memoria principale
- il programma (o una parte di esso) può essere **caricato in una qualsiasi zona di memoria** principale sufficientemente grande per contenerlo
- il programma **può essere spostato rilocazione**



Deve esistere un meccanismo in grado di convertire gli **indirizzi logici** (I_l) contenuti nelle istruzioni del programma,
in corrispondenti **indirizzi fisici** (I_f)

Programma caricato completamente in memoria:

$$I_f = I_l + I_b$$

I_b è l'indirizzo fisico della locazione di memoria che, dopo il caricamento, contiene la prima istruzione (o dato) del programma (*indirizzo base* o *fattore di rilocazione*)

Conversione software degli indirizzi:

- il *loader* somma ad ogni indirizzo logico il valore di I_b
- al momento dell'esecuzione, le istruzioni del programma contengono indirizzi assoluti e l'eventuale rilocazione del programma può avvenire solo se tali indirizzi vengono ricalcolati

Conversione hardware degli indirizzi:

- registro dedicato che contiene il valore di I_b del programma in esecuzione
- il valore corrente del registro dedicato viene automaticamente sommato agli indirizzi logici contenuti nelle istruzioni **che non cambiano mai**
- rilocazione dinamica a costo zero: è sufficiente modificare il valore del registro dedicato

☞ Esecuzione in presenza di un sistema operativo
⇒ le istruzioni devono contenere indirizzi logici con poche eccezioni...

INDIRIZZAMENTO DIRETTO

I_{eff} dell'operando **contenuto nell'istruzione**

INDIRIZZAMENTO INDIRETTO (o DIFFERITO)

I_{eff} dell'operando **contenuto**
nella locazione di memoria indirizzata
dal campo operando dell'istruzione

L'indirizzamento indiretto *costa* un ciclo di memoria in più, rispetto all'indirizzamento diretto:

- sono necessari **due accessi alla memoria**:
  - il primo per **leggere l'indirizzo dell'operando**
  - il secondo per **accedere all'operando**

INDIRIZZAMENTO MEDIANTE REGISTRO

(*referred addressing* o *pointer addressing*)

I_{eff} viene calcolato in funzione di uno o più registri

Tecnica più semplice:
un registro contiene l'indirizzo dell'operando dell'istruzione

Tecniche più complesse:
I_{eff} = **registro base + registro indice** +
spiazzamento costante (*displacement*)

Valore dello spiazzamento
- **senza segno**
- **con segno** (*spiazzamento relativo*)

Un solo accesso alla memoria

INDIRIZZAMENTO MEDIANTE REGISTRO BASE

- un **registro base** contiene l'**indirizzo** di partenza di un'area di memoria in cui sono memorizzati dati o istruzioni
- lo **spiazzamento** (di norma senza segno) permette di accedere ai vari elementi di tale area

$$I_{eff} = \text{RegistroBase} + \text{Spiazzamento}$$

Dimensione dell'area indirizzabile
dipende dalla dimensione (in bit) del campo in cui viene memorizzato lo spiazzamento

INDIRIZZAMENTO MEDIANTE REGISTRO INDICE

- il campo relativo allo **spiazzamento** contiene, in realtà, l'**indirizzo** di partenza di un'area di memoria in cui è memorizzata una struttura dati (vettore)
- un **registro indice** permette di accedere ai vari elementi della struttura (del vettore)

$$I_{eff} = \text{IndirizzoBase} + \text{RegistroIndice}$$

INDIRIZZAMENTO RELATIVO AL PROGRAM COUNTER

Istruzioni di salto $\Rightarrow$
operando = istruzione a cui passare il controllo

In genere, il campo operando di tali istruzioni contiene
- non l'indirizzo cui saltare, bensì
- uno **spiazzamento relativo** (cioè con segno) rispetto al valore corrente del *program counter*

$$I_{eff} = PC + SpiazzamentoRelativo$$

PC già incrementato $\Rightarrow$ contiene l'indirizzo dell'istruzione successiva a quella in esecuzione

Operandi delle istruzioni di salto **indipendenti dalla posizione** del programma in memoria

INDIRIZZAMENTO IMMEDIATO

Operando (valore costante) nell'istruzione

ADD AR, 120; AR $\leftarrow$ AR + 120

- 120 in una locazione di memoria + indirizzo di tale locazione nell'istruzione ADD
- 120 inserito direttamente nell'istruzione ADD

Vantaggio
il registro IR contiene il valore dell'operando e quindi non è necessario un ulteriore accesso alla memoria

- la costante può essere contenuta nell'istruzione?
- codici operativi diversi:
  - ADD AR, Var ; indirizzo
  - ADD AR, 120; costante

ARCHITETTURE A REGISTRI GENERALI

CPU in stato di attesa se operando sorgente in memoria principale

Velocità di elaborazione della CPU limitata dalla velocità della memoria principale

Lo stesso tipo di istruzione viene eseguita
- più velocemente se coinvolge solo registri macchina
- più lentamente se coinvolge una o più locazioni di memoria principale

Tempo di esecuzione dell'istruzione direttamente **proporzionale al numero di accessi alla memoria**

Due tecniche per migliorare le prestazioni della CPU:
- **utilizzare memoria *cache***
- **aumentare** il **numero** dei **registri** macchina

Tecniche indipendenti e quindi utilizzabili nella stessa CPU

Memoria *cache*

- nella CPU, oppure esterna
- **del tutto invisibile all'utilizzatore** - non esistono istruzioni per accedere alla memoria *cache*
- contiene una copia di una porzione contigua di memoria principale (codice e/o dati)
- permette un accesso più veloce alle informazioni in essa memorizzate

Incremento del numero dei registri macchina

- è esternamente **visibile** - le istruzioni macchina devono poter accedere ai vari registri esistenti

Registri organizzati in una piccola memoria, interna alla CPU, i cui elementi (i registri) sono
- **indirizzabili direttamente**
- **utilizzabili indifferentemente** - qualche eccezione

Ogni registro
- può contenere un **valore** o un **indirizzo**
- può essere utilizzato come **registro base** o come **registro indice**

Architettura a **registri generali**
Elaboratori con 8, 16, o più registri generali
Numero essenzialmente limitato dal costo

CISC e RISC

Elaboratori a registri generali di due tipi:
- **Complex Instruction-Set Computer** (CISC)
- **Reduced Instruction-Set Computer** (RISC)

Differenze:
- numero e complessità delle istruzioni macchina
- diverse strategie per accedere agli operandi in memoria principale

Classico esempio di **CISC** (Intel, Motorola):
- grande varietà di tipi di dati e di modalità di indirizzamento
- insieme esteso di istruzioni
- possibilità di accedere a più operandi in memoria nella stessa istruzione (un'unica istruzione permette di sommare due operandi in memoria e di memorizzare il risultato in memoria - 3 accessi)

Maggiore complessità nella realizzazione dell'unità di controllo
Esecuzione delle istruzioni macchina comandata da microcodice e determinata di volta in volta e in base al tipo di operandi coinvolti
Maggiore flessibilità (modifica delle microistruzioni)
Minori prestazioni globali

Un **RISC** non permette alle istruzioni macchina che agiscono sull'ALU o sulla FPU (Floating Point Unit) di accedere direttamente alla memoria

Le istruzioni possono accedere solo ai registri

L'accesso alla memoria avviene esclusivamente mediante istruzioni di ***load*** e ***store***

Semplificazione del progetto dell'unità di controllo (non più microprogrammata)
Aumento delle prestazioni globali

Attualmente, nonostante i vantaggi offerti dai RISC, è predominante il numero di CISC sul mercato
(gli Intel 80x86 sono CISC)

La tendenza è quella di progettare nuovi CISC, compatibili con i precedenti, che includono tipiche scelte RISC
ad esempio, alcune istruzioni macchina del 486 e del Pentium vengono eseguite senza microcodice

DISPOSITIVI DI INGRESSO / USCITA

Esterni alla CPU
Parte del controllo dell'I/O può essere nella CPU

- **dispositivi di ingresso**: tastiere, mouse, scanner, dispositivi per input vocale
- **dispositivi di uscita**: schermi alfanumerici e/o grafici, stampanti, dispositivi per output vocale
- **dispositivi di ingresso e uscita**: reti di comunicazione
- **dispositivi di memorizzazione**: nastri, dischi magnetici e ottici - (le memorie secondarie sono considerate a tutti gli effetti dispositivi di I/O e come tali vengono gestite)

Tecniche di comunicazione tra CPU e dispositivi:

- **I/O mappato in memoria** -
  locazioni di memoria principale, di indirizzi fissi, assegnate permanentemente al dispositivo di I/O
  letture e/o scritture su tali locazioni hanno automaticamente effetto sul dispositivo di I/O
- **I/O non mappato in memoria** - speciali istruzioni di I/O per la lettura (**IN**put) e la scrittura (**OUT**put)
  spazio di indirizzamento dell'I/O
  al dispositivo di I/O vengono assegnate 1+ **porte**

Nel primo caso:
- **tutte le istruzioni** (che accettano come operando un indirizzo di memoria) **possono essere utilizzate per eseguire dell'I/O**
- deve esistere una **gestione hardware** delle locazioni di memoria riservate ai dispositivi di I/O

VAX - solo I/O mappato in memoria
⇒ **non esistono istruzioni dedicate di I/O**

Esistono dispositivi di I/O in cui
- per alcune funzioni si utilizzano indirizzi di memoria
- per altre funzioni si utilizzano indirizzi di I/O

Esempio: **adattatore video**
- visualizzazione dati sullo schermo mappato in memoria: *bit map* (array di byte che descrive i punti, o pixel, visualizzati sullo schermo) può occupare 16k, 32k, 256k o più byte di memoria principale
- modifica della modalità grafica con istruzioni di I/O

Video alfanumerico $24 * 80$ caratteri

b/w ➨ 1920 byte

Video grafico $480 * 640$ pixel

b/w ➨ 38400 byte
256 colori ➨ 307200 byte
16M colori ➨ 921600 byte

OPERAZIONI SUI DISPOSITIVI DI I/O

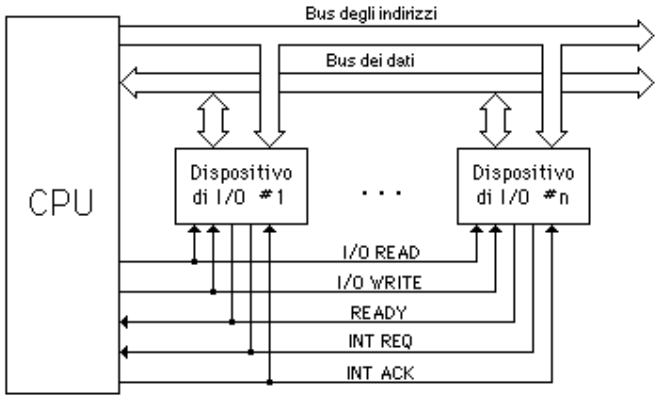
Operazioni logiche su dispositivo di I/O:
- **lettura di un dato** da una porta di input;
- **lettura dello stato** del dispositivo da una porta di controllo;
- **invio di un dato** su una porta di output;
- **invio di un comando** su una porta di controllo.

Esempio - Stampante con
una porta di output e una porta di controllo
- i dati inviati alla porta di output vengono stampati
- i comandi inviati alla porta di controllo vengono eseguiti

Mediante comandi la CPU può
- chiedere lo *stato* del dispositivo
- *attivare* o *disattivare* il dispositivo
- iniziare un'operazione di trasferimento dati

In pratica:
- operazioni di **input** - dato o stato del dispositivo
- operazioni di **output** - dato o comando

Bus degli indirizzi + bus dati +
linee di controllo I/O READ, I/O WRITE e READY



Input:

❶ **CPU** - bus degli indirizzi = indirizzo porta di I/O

❷ **CPU** - richiede un'operazione di input ⇒ segnale sulla linea I/O READ

❸ **dispositivo di I/O** - carica l'informazione richiesta sul bus dei dati

❹ **dispositivo di I/O** - indica che l'informazione è sul bus dei dati ⇒ segnale sulla linea READY

Output:

❶ **CPU** - bus degli indirizzi = indirizzo porta di I/O; bus dei dati = dato da inviare

❷ **CPU** - richiede un'operazione di output ⇒ segnale sulla linea I/O WRITE

❸ **dispositivo di I/O** - legge il dato contenuto nel bus dei dati

❹ **dispositivo di I/O** - indica la fine dell'operazione (disponibilità dei bus) ⇒ segnale sulla linea READY

GESTIONE DEI DISPOSITIVI DI I/O

Caratteristiche dei dispositivi di I/O:

- **funzionamento** completamente **asincrono** rispetto a quello dell'unità centrale
- **velocità inferiore** sia a quella dell'unità centrale (spesso di alcuni ordini di grandezza), sia a quella della memoria principale
- **formato dei dati**, in genere, **differente** da quello utilizzato nell'unità centrale

Gestione dei dispositivi di I/O:

- software
  - program-driven
- hardware
  - interrupt
  - DMA - Direct Memory Access

GESTIONE SOFTWARE - POLLING

Il programma deve controllare periodicamente lo stato di ogni dispositivo (**polling**),
al fine di accertarsi se è possibile (o necessario) eseguire un'operazione di I/O - *servire* il dispositivo

Per ogni dispositivo di input, la CPU deve:

❶ richiedere lo stato del dispositivo
❷ verificare se sul dispositivo è disponibile un dato
❸ in caso affermativo, eseguire l'input del dato

Per ogni dispositivo di output, la CPU deve:

❶ richiedere lo stato del dispositivo
❷ verificare se il dispositivo è pronto a ricevere un dato
❸ in caso affermativo, inviare al dispositivo il dato e, se necessario, il comando di effettuare l'output

Per effettuare l'input o l'output di ogni dato è necessario **eseguire un** apposito **sottoprogramma**

Vantaggi:

- non è richiesto hardware aggiuntivo

Svantaggi:

- notevole consumo di tempo di calcolo
  - richiesta e verifica periodica dello stato di tutti i dispositivi, compresi quelli che
    - non hanno bisogno di essere serviti
    - non sono ancora pronti
  - intervallo di tempo tra due controlli successivi funzione della velocità del dispositivo da gestire: più il dispositivo è veloce, più i controlli devono essere frequenti - possibilità di perdere dati in ingresso
- se si aggiunge un dispositivo di I/O, è necessario modificare il programma di gestione delle periferiche

Può non garantire tempi di risposta sufficientemente rapidi per il servizio dei dispositivi molto veloci (dischi magnetici), oppure
può arrivare a monopolizzare la CPU per troppo tempo

GESTIONE HARDWARE - INTERRUZIONI

Hardware dedicato
sovrapposizione elaborazioni CPU e I/O

La CPU **inizia un'operazione di I/O** (una lettura) e quindi **riprende la sua elaborazione**
Quando il dato da leggere è pronto, il dispositivo invia alla CPU una **richiesta di servizio** (**interrupt**)
A questo punto, la CPU può trasferire il dato dal dispositivo alla memoria principale

Quando riceve un interrupt la CPU deve:
- **accorgersi** che esiste una **richiesta** di servizio
  in genere, solo **dopo la fase di execute** dell'istruzione corrente
- **identificare il tipo** di segnale di interrupt e decidere se soddisfare oppure no la richiesta - possibilità di disabilitare il sistema di interruzione esterno
- **servire la richiesta** - interrompendo, in modo del tutto trasparente, il programma in esecuzione e passando il controllo ad una opportuna **routine di servizio** (gestore) dell'interrupt

Tutto in hardware

Attivazione della routine di servizio dell'interrupt dipende dall'architettura

Caso più semplice:
unico sottoprogramma, in posizione fissa, che
- identifica il tipo di sorgente e il tipo di richiesta
- chiama la routine di servizio opportuna

Tecnica più sofisticata:
vettore, in posizione fissa, **contenente gli indirizzi delle routine di servizio**
Se la CPU accetta di servire l'interrupt,
il dispositivo interessato invia sul bus dei dati un **indice** che permette di selezionare (in hardware) un elemento del vettore (cioè una routine di servizio)

Elaboratore ad **interrupt vettorizzati**

Ogni routine di servizio dell'interrupt deve:

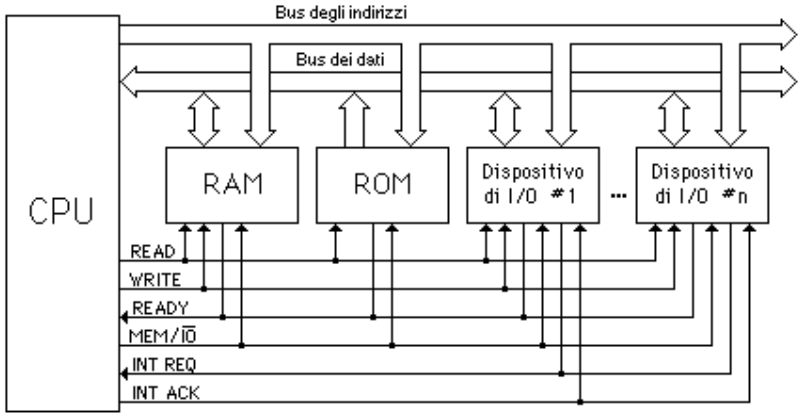
❶ salvare il contenuto di tutti i registri utilizzati durante il servizio

❷ acquisire, se necessario, tutte le informazioni possibili sulla natura dell'interruzione - lo stesso dispositivo può lanciare un interrupt per motivi diversi

❸ eseguire le azioni appropriate per gestire la condizione di interruzione - ad esempio:
- trasferire uno o più dati
- inviare i comandi necessari per eseguire una nuova operazione
- dichiarare chiusa la sequenza di operazioni

❹ ripristinare lo stato completo del programma interrotto e riattivarlo

Il programmatore deve decidere se l'esecuzione può avvenire ad interrupt abilitati (la routine di servizio è interrompibile), oppure no



- INT REQ (INTerrupt REQuest) - utilizzata dai dispositivi periferici per inviare all'unità centrale una richiesta di interruzione

- INT ACK (INTerrupt ACKnowledge) - utilizzata dalla CPU per segnalare al dispositivo che l'interrupt è stato preso in considerazione e per richiedere, nel caso di interrupt vettorizzati, l'invio da parte del dispositivo del tipo di richiesta (indice nel vettore)

Per ogni singolo trasferimento la CPU deve:
- interrompere il programma in esecuzione
- passare il controllo alla routine di servizio
- ripassare il controllo al programma interrotto

Costo elevato
in termini di tempo e di occupazione della CPU

In alcuni casi, i trasferimenti di dati dai dispositivi di I/O alla memoria principale, o viceversa, possono essere troppo rapidi per i tempi di esecuzione delle istruzioni della CPU e impedire ogni altra attività

GESTIONE HARDWARE - DMA

Per ridurre
- il tempo necessario per il trasferimento dei dati
- il tempo necessario per la relativa gestione da parte della CPU

i dispositivi periferici devono poter accedere direttamente alla memoria principale

Tecnica del **DMA** (Direct Memory Access)

Controllore dedicato al trasferimento dei dati, che interferisce il meno possibile con la CPU

Il controllore del DMA esegue (in hardware)
il **trasferimento di un intero blocco di dati** tra dispositivi di I/O e memoria principale,
senza che i dati passino attraverso l'unità centrale

La CPU è coinvolta solo
all'inizio e al termine del trasferimento del blocco

Esiste già un **percorso diretto**, utilizzabile per questi trasferimenti: **i bus dei dati e degli indirizzi**, in comune tra memoria principale e dispositivi di I/O

Il dispositivo di I/O, anziché inviare direttamente un interrupt alla CPU, richiede un accesso in memoria al controllore del DMA, il quale **assume il controllo dei bus del sistema** ed effettua il trasferimento

MICROPROCESSORI

Novembre 1971
Intel lancia sul mercato la CPU 4004
"*elaboratore micro-programmabile su un unico chip*"
16 piedini, 2300 transistor, 46 istruzioni,
4096 byte di memoria

Vera e propria rivoluzione
piccole dimensioni e **basso costo**
⇒ potenza dei calcolatori disponibile a un vasto numero di applicazioni

Prima del microprocessore (*special-purpose*)
hardware dedicato, in grado di risolvere solo i problemi specifici di ciascuna diversa applicazione
logica cablata, circuiti logici tra loro permanentemente connessi

Con il microprocessore (*general-purpose*)
unità standard programmabili,
prodotte in grande quantità e quindi a basso costo, permettono di risolvere un gran numero di problemi diversi mediante la semplice modifica di programmi

Vantaggi:
- **riduzione dell'hardware**
  in numero e tipo di circuiti integrati
- **riduzione del costo** del prodotto finale
- **maggiore flessibilità**
  possibilità di modificare il software
- **maggiore affidabilità**
  circuiti realizzati con tecniche sofisticate

Microprocessore =
CPU in un **unico circuito integrato** (*chip*)

Microcomputer =
µP + memoria principale + controllo dell'I/O

Struttura identica a quella di un elaboratore classico

Memoria principale di varie dimensioni
(da pochi Kbyte a molti Mbyte) costituita da:
- **RAM** (Random Access Memory)
- **ROM** (Read-Only Memory)

Interfaccia con il mondo esterno:
insieme di elementi - **porte di I/O**
nello stesso chip del µP, oppure esterne